EDUBRIDGE AFRICA · TRACED FROM THE CODE ON MAIN @ 69E950B

How a parent and a school actually use EduBridge

There is no app to download. EduBridge is a website a parent opens from a link on any phone browser, and SMS does the rest. Below is every step from first contact to daily use, marked by what the code does today.

🟢 Works 🟡 Partial or needs setup 🔴 Not built / blocks the journey

A parent cannot get in today

When an admin adds a student, the parent's phone number is saved on the student record only. No parent account is ever created and the student is never linked to one. So at step 3 a parent gets "Account not found — contact your school admin", and even if one got through, their dashboard would show no children. Separately, every alert SMS is queued but nothing sends the queue.

JOURNEY A

Parent

A guardian with a basic smartphone and a Ugandan number.

GETTING IN

1 **School registers the parent** 🔴 Not built

Admin adds the child with a parent phone number. That number is stored on the student, but no parent login account is made and `students.parent_id` stays empty.

`api/students/route.ts`

2 **Parent gets the link** 🟡 Manual

Opens the EduBridge web address in the phone browser and taps "Log in". Nothing sends the link today; the school would have to share it by hand.

No install. There is also no home-screen icon setup (no web app manifest) yet.

`app/page.tsx`

3 **Enters phone number** 🔴 Blocked by 1

The server checks the number belongs to an active account. No self-signup, by design.

> Not registered → "Account not found — contact your school admin". This is where every parent stops today.

`/login → api/auth/request-otp`

4 **Receives a login code by SMS** 🟡 Needs config

Supabase generates the code; a webhook sends it through Africa's Talking: "Your EduBridge Africa login code is 123456".

`api/auth/sms-hook`

5 **Enters the code** 🟢 Works

Code verified, account linked to the phone on first login, then sent to the parent home by role.

> Wrong or expired code → "Invalid or expired code", try again.

`/verify → api/auth/verify-otp`

EVERYDAY USE

6 **Parent dashboard** 🔴 Empty

Lists the parent's children by looking up students where `parent_id` is this parent. Since step 1 never sets it, the list is empty.

`/parent · lib/supabase/parent.ts`

7 **Checks on the child** 🟢 Built

🟢 Attendance 🟢 Grades + class average 🟢 Fees & balance 🟢 Announcements 🔴 Daily reports 🔴 Incidents

Daily reports and Incidents are titled placeholder pages (Phase 2).

8 **Gets SMS alerts without logging in** 🔴 Never sent

Absences, new grades, payments and announcements each queue an SMS as "pending". The worker that should send them returns "Not implemented", and no cron schedule is set in `vercel.json`.

`lib/notifications/queue.ts · api/cron/process-sms`

JOURNEY B

School admin

Head teacher or office admin. A bursar logs in the same way and lands on Fees.

ONBOARDING THE SCHOOL

1 **School and first admin created** 🟡 By hand in SQL

There is no school signup screen and the super-admin schools API is a stub. EduBridge staff insert the school and the admin's phone in the database.

`api/schools (501) · supabase/migrations`

2 **Admin logs in** 🟢 Works

Same phone and SMS-code login as parents, then lands on `/admin` (bursar on `/admin/fees`).

3 **Sets up the term** 🟡 Mostly built

🟢 Classes 🟢 Teachers (creates their logins) 🟢 Students, one at a time 🔴 CSV student import 🟢 Fee structure

Teachers get real accounts here; parents don't. Adding 600 students one by one is the practical onboarding wall.

RUNNING THE SCHOOL DAY

4 **Teachers record attendance and grades** 🟢 Built

Teachers mark each class register and fill the gradebook; the admin sees school-wide overviews.

`/teacher/attendance · /teacher/grades · /admin/attendance · /admin/grades`

5 **Bursar records fee payments** 🟢 Built

Each payment updates the student's balance and queues a receipt SMS. Weekly overdue reminders are still a stub.

`/admin/fees · api/cron/fee-reminders (501)`

6 **Sends an announcement** 🟢 Built

Posts to the parent portal, with an option to also SMS every parent.

`/admin/announcements`

7 **Checks system health** 🟡 Shows the gap

Lists the SMS queue. Today it would show every message stuck at "pending".

`/admin/system`

WHERE THE TWO JOURNEYS MEET

What the parent sees when the school acts

The product promise is the right-hand column: parents hear about it by SMS without opening anything. That channel is the part that doesn't run yet.

SCHOOL DOES	IN THE PARENT PORTAL	BY SMS TO THE PARENT
Teacher marks a child absent	🟢 Shows once the parent is linked	🔴 Queued, never sent api/attendance
Teacher submits grades	🟢 Shows, with class average	🔴 Queued, never sent api/grades
Bursar records a payment	🟢 Balance updates	🔴 Queued, never sent api/fees/payments
Fees go overdue	🟢 Balance shows	🔴 Reminder job not built api/cron/fee-reminders
Admin posts an announcement	🟢 Shows	🔴 Queued, never sent api/announcements
Admin sends a one-off SMS	🟡 n/a	🔴 Not built api/sms/send (501)

TO MAKE JOURNEY A WORK END TO END

Three changes, in this order

FIRST

Create and link parent accounts

When a student is saved with a parent phone, find or create a parent user for that number and set `parent_id`. One parent, many children, matched by phone. Without this nobody gets past login.

SECOND

Send the SMS queue

Implement `process-sms` to send pending rows through Africa's Talking, mark them sent or failed, and add the cron schedule to `vercel.json`. This is the feature parents notice most.

THIRD

Welcome SMS with the link

When a parent account is created, text them the web address and how to log in. That replaces the "download" moment and removes step 2's manual handoff.

Traced from the repository source: routes, API handlers and migrations. Build, lint and typecheck pass; nothing here was run against a live Supabase project or Africa's Talking account.